

UNIVERZA V LJUBLJANI
FAKULTETA ZA RAČUNALNIŠTVO IN INFORMATIKO

Nik Katanović Leban

**Spletna aplikacija za prodajo in
evalvacijo avtomobilov**

DIPLOMSKO DELO

VISOKOŠOLSKI STROKOVNI ŠTUDIJSKI PROGRAM
PRVE STOPNJE

MENTOR: viš. pred. dr. Igor Rožanc

Ljubljana, 2026

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati diplomskega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.



Izvorna koda diplomskega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani <http://www.gnu.org/licenses/>.

Besedilo je oblikovano z urejevalnikom besedil $\text{\LaTeX}$.

Kandidat: Nik Katanović Leban

Naslov: Spletna aplikacija za prodajo in evalvacijo avtomobilov

Vrsta naloge: Diplomski naloga na visokošolskem programu prve stopnje
Računalništvo in informatika

Mentor: viš. pred. dr. Igor Rožanc

Opis:

Besedilo teme diplomskega dela študent prepíše iz študijskega informacijskega sistema, kamor ga je vnesel mentor. V nekaj stavkih bo opisal, kaj pričakuje od kandidatovega diplomskega dela. Kaj so cilji, kakšne metode naj uporabi, morda bo zapisal tudi ključno literaturo.

Title: Web application for car sales and evaluation

Description:

The student shall transcribe the text of the diploma thesis topic from the student information system, where it was entered by the mentor. In a few sentences, they will describe what is expected of the candidate's thesis. What the objectives are, what methods should be used, and perhaps they will also list the key literature.

Rad bi se zahvalil svojemu mentorju Igorju Rožancu, za vso aktivno pomoč in svetovanje pri razvoju te diplomske naloge. Zahvalil bi se tudi vsem sodelavcem iz podjetja CargoX, ki so z mano delili svoje znanje in izkušnje, ter mi omogočili, da sem se lahko lotil tako zahtevnega projekta. Posebna zahvala gre moji družini, ki me je ves čas podpirala in mi stala ob strani.

Kazalo

Povzetek

Abstract

1	Uvod	1
1.1	Opis arhitekture sistema	2
2	Pregled področja in sorodnih del	5
3	Tematsko ozadje	7
3.1	Spletne aplikacije	7
3.2	Relacijske podatkovne baze	8
3.3	Evalvacijski sistem	9
3.4	Mikrostoritvena arhitektura	9
4	Struktura diplomske naloge	11
4.1	Slogovne smernice in priporočila	12
5	Osnovni gradniki L^AT_EXa in nasveti za pisanje	15
5.1	Koristni nasveti pri pisanju v L ^A T _E Xu	16
5.2	Pisave v L ^A T _E Xu	17
5.3	Skladnost s standardom PDF/A	17
5.4	Uporaba količin in enot	18
5.5	Sklici in citiranje	19

6	Pogoste napake pri pisanju v slovenščini	21
7	Plovke: slike in tabele	25
7.1	Formati slik	26
7.2	Podnapisi k slikam in tabelam	27
7.3	Vključevanje več slik in tabel hkrati	27
8	Vključevanje programske kode in algoritmov	33
8.1	Vključevanje algoritmov (psevdokode)	34
9	Matematično okolje in sklicevanje na besedilne konstrukte	37
9.1	Matrike	38
9.2	Razbijanje dolgih enačb	39
10	Kaj pa literatura?	41
11	Sklepne ugotovitve	43
	Literatura	45
	Ostala literatura	45

Povzetek

Naslov: Spletna aplikacija za prodajo in evalvacijo avtomobilov

Avtor: Nik Katanović Leban

Diplomsko delo obravnava problem določanja primerne tržne vrednosti rabljenega avtomobila. Pri prodaji ali nakupu vozila uporabniki pogosto nimajo zadostnega pregleda nad trenutnim stanjem na trgu, zato težko ocenijo, ali je določena cena primerna. Posledično lahko vozilo kupijo po previsoki ceni ali ga prodajo pod njegovo dejansko tržno vrednostjo. Za reševanje tega problema sem razvil spletno aplikacijo za objavo in pregled avtomobilskih oglasov, ki uporabnikom omogoča enostavno in varno prodajo ter nakup vozil. Poleg tega je bil implementiran sistem za avtomatsko ocenjevanje vrednosti vozil. Sistem temelji na zbiranju podatkov iz zunanjih virov, njihovi normalizaciji in primerjavi vozila s podobnimi vozili na trgu. Pri oceni vrednosti se upoštevajo ključne lastnosti vozila, kot so znamka, model, letnik in prevoženi kilometri. Za pridobivanje podatkov so bili uporabljeni javno dostopni podatkovni viri ter postopki samodejnega zajema podatkov s spletnih strani za prodajo vozil. Zbrane podatke sistem obdela in uporabi za izračun ocenjene tržne vrednosti vozila. Rezultat diplomskega dela je delujoča spletna aplikacija s podporo za avtomatsko vrednotenje vozil, ki uporabnikom pomaga pri sprejemanju bolj informiranih odločitev pri nakupu ali prodaji avtomobila.

Ključne besede: spletna aplikacija, avtomobili, vrednotenje, React, Express.js, Node.js, PostgreSQL, REST API, Docker, GitHub Actions, TypeScript, Python, OAuth2, CI/CD.

Abstract

Title: Web application for car sales and evaluation

Author: Nik Katanović Leban

This work addresses the problem of determining the appropriate market value of a used vehicle. When buying or selling a car, users often lack sufficient insight into current market conditions, making it difficult to assess whether a listed price is fair. As a result, vehicles may be purchased at inflated prices or sold below their actual market value. To address this problem, a web application for publishing and browsing vehicle listings was developed, enabling users to buy and sell vehicles in a simple and secure manner. In addition, an automated vehicle valuation system was implemented. The system is based on collecting data from external sources, normalizing the acquired data, and comparing a vehicle with similar vehicles currently available on the market. The valuation process takes into account key vehicle characteristics, including brand, model, year of first registration, and mileage. The required data was obtained from publicly available datasets and through automated data collection from vehicle marketplace websites. The collected data is processed and used to estimate the market value of a vehicle. The result of this thesis is a functional web application with integrated vehicle valuation support that helps users make more informed decisions when buying or selling used vehicles.

Keywords: web application, automobiles, evaluation, React, Express.js, Node.js, PostgreSQL, REST API, Docker, GitHub Actions, TypeScript, Python, OAuth2, CI/CD.

Poglavje 1

Uvod

Nakup in prodaja rabljenih avtomobilov predstavljata pomemben del avtomobilskega trga. Pri tem se uporabniki pogosto srečujejo s težavo določanja primerne tržne vrednosti vozila, saj nimajo zadostnega pregleda nad trenutno ponudbo in gibanjem cen na trgu. Najpogostejši pristop je primerjava z obstoječimi oglasi podobnih vozil, vendar takšna metoda ni vedno zanesljiva. Oglasi lahko vsebujejo nerealno visoke ali nizke cene, uporabniki pa pogosto nimajo dovolj znanja in izkušenj, da bi lahko ocenili dejansko vrednost vozila. Posledično lahko vozilo prodajo pod njegovo tržno vrednostjo ali ga kupijo po previsoki ceni.

Na trgu že obstajajo rešitve za ocenjevanje vrednosti vozil, vendar so mnoge plačljive, prilagojene tujim trgom ali pa temeljijo na zastarelih podatkih. Zaradi tega njihove ocene pogosto ne odražajo dejanskega stanja na slovenskem trgu rabljenih vozil.

Cilj diplomskega dela je razvoj spletne aplikacije za objavo in pregled avtomobilskih oglasov ter razvoj sistema za samodejno ocenjevanje vrednosti vozil. Sistem temelji na zbiranju podatkov iz zunanjih virov, njihovi normalizaciji in analizi primerljivih vozil na trgu. Pri določanju ocenjene vrednosti se upoštevajo ključne lastnosti vozila, kot so znamka, model, letnik prve re-

gistracije in število prevoženih kilometrov.

Hipoteza diplomskega dela je, da je mogoče s pomočjo zbranih tržnih podatkov in primerjalne analize podobnih vozil uporabniku ponuditi uporabno in dovolj natančno oceno tržne vrednosti vozila. Dodatni cilj je preveriti, ali integracija takšnega sistema v spletno aplikacijo izboljša uporabniško izkušnjo pri nakupu in prodaji vozil.

V poglavju ?? je predstavljena analiza problema, pregled obstoječih rešitev in uporabljene tehnologije. Poglavje ?? opisuje načrtovanje sistema in arhitekturo aplikacije. Implementacija posameznih komponent, vključno s sistemom za vrednotenje vozil, je predstavljena v poglavju ?. V poglavju ?? so opisani postopki testiranja in rezultati evalvacije sistema. Diplomsko delo se zaključuje s sklepom in predlogi za nadaljnje izboljšave.

1.1 Opis arhitekture sistema

Spletna aplikacija je sestavljena iz uporabniškega vmesnika (odjemalec), strežniškega dela aplikacije (zaledni sistem) in ločenega sistema za vrednotenje vozil. Uporabniški vmesnik je razvit z uporabo knjižnice React in programskega jezika TypeScript v okolju Node.js različice 24. Zaledni sistem je razvit z uporabo ogrodja Express v programskem jeziku JavaScript v okolju Node.js različice 24. Sistem za vrednotenje vozil je implementiran kot ločena storitev v programskem jeziku Python različice 3.11.

Za izvajanje posameznih komponent je uporabljena platforma Docker.

Docker kontejnerji:

- Zaledni sistem
- Postgres:16
- redis:7
- mailhog
- Evalvacijski sistem

Razvoj aplikacije je potekal v razvojnem okolju Visual Studio Code. Za avtomatizacijo preverjanja kode in izvajanja testov je bil uporabljen sistem GitHub Actions, medtem ko so se periodična opravila izvajala s pomočjo časovno načrtovanih opravil (cron). Za gostovanje aplikacije je bil uporabljen virtualni strežnik ponudnika Hetzner. Namestitev in upravljanje aplikacije na strežniku je bilo izvedeno s pomočjo platforme Dokploy.

Poglavje 2

Pregled področja in sorodnih del

Na področju prodaje rabljenih vozil obstaja več spletnih aplikacij, ki uporabnikom omogočajo objavo in pregled avtomobilskih oglasov. Med najbolj znanimi evropskimi ponudniki sta mobile.de in AutoScout24, ki združujeta veliko število oglasov iz različnih držav ter uporabnikom omogočata filtriranje vozil po številnih kriterijih. Za slovenski trg je najbolj razširjena platforma Avto.net, ki predstavlja osrednje mesto za objavo in iskanje rabljenih vozil. Nekatere izmed omenjenih platform uporabnikom ponujajo tudi osnovno oceno primernosti cene vozila. Mobile.de na primer pri posameznih oglasih prikazuje informacijo o tem, ali je cena vozila glede na primerljive oglase ugodna, primerna ali previsoka. Takšen pristop uporabnikom omogoča hitrejšo orientacijo na trgu, vendar praviloma ne ponuja možnosti samostojnega vrednotenja vozila na podlagi uporabniško vnesenih podatkov.

Poleg spletnih tržnic obstajajo tudi specializirane storitve za vrednotenje vozil. Med najbolj znanimi je Eurotax, ki se pogosto uporablja v avtomobilski industriji, zavarovalništvu in pri trgovcih z vozili. Njegove ocene temeljijo na obsežnih zbirkah podatkov o vozilih in tržnih cenah, vendar je dostop do storitve praviloma plačljiv. Zaradi poslovnega modela in omejenega dostopa podrobna analiza delovanja sistema ni mogoča. Na ameriškem trgu sta razširjeni storitvi Kelley Blue Book in CARFAX. Kelley Blue Book uporabnikom omogoča oceno vrednosti vozila na podlagi tehničnih podatkov in

tržnih analiz, medtem ko CARFAX poleg vrednotenja ponuja tudi vpogled v zgodovino vozila. Obe storitvi sta prilagojeni predvsem severnoameriškemu trgu in zato nista neposredno uporabni za slovenski trg rabljenih vozil.

Pregled obstoječih rešitev je pokazal, da večina platform bodisi omogoča prodajo vozil brez naprednega sistema za vrednotenje bodisi ponuja ločene plačljive storitve za ocenjevanje vrednosti vozil. Namen razvite rešitve je združiti obe funkcionalnosti v enotni spletni aplikaciji, ki uporabnikom omogoča objavo vozil, pregled trga in samodejno oceno vrednosti vozila na podlagi primerljivih vozil, dostopnih na trgu.

Poglavje 3

Tematsko ozadje

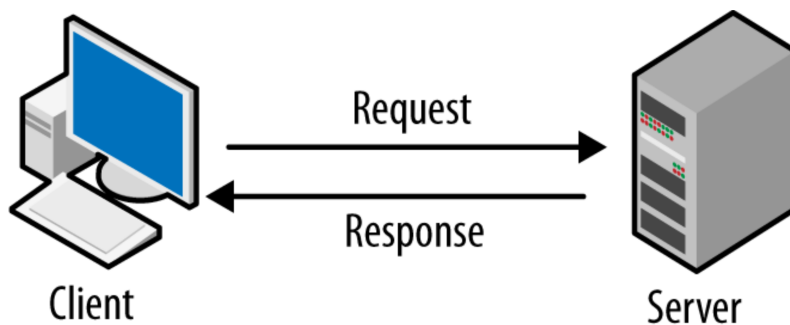
3.1 Spletne aplikacije

Spletne aplikacije sestavljata uporabniški vmesnik (odjemalec) in strežniški del aplikacije (zaledni sistem). Takšna arhitektura omogoča ločitev skrbi, kjer vsak del sistema prevzame svojo vlogo v celotni aplikaciji [richardson2007restful].

Uporabniški vmesnik omogoča prikaz podatkov ter interakcijo uporabnika z aplikacijo. Njegova glavna naloga je zagotavljanje preglednega, odzivnega in uporabniku prijaznega dostopa do funkcionalnosti sistema. Poleg prikaza podatkov skrbi tudi za zajem uporabniških vnosov, komunikacijo s strežniškim delom aplikacije ter lokalno shranjevanje določenih podatkov v spletnem brskalniku, kar prispeva k boljši odzivnosti in uporabniški izkušnji [flanagan2020javascript]. Sodobni uporabniški vmesniki so pogosto zasnovani na komponentni arhitekturi. Komponenta predstavlja samostojen in ponovno uporaben gradnik uporabniškega vmesnika, ki ga je mogoče vključiti na različnih mestih v aplikaciji. Takšen pristop izboljšuje modularnost sistema, poenostavlja vzdrževanje kode ter omogoča boljšo ponovno uporabo funkcionalnosti [react_component_model].

Strežniški del aplikacije je odgovoren za obdelavo zahtevkov, ki jih prejme od odjemalca, ter za vračanje ustreznih odgovorov. Poleg tega izvaja po-

slovno logiko sistema, upravlja z dostopnimi pravicami uporabnikov ter skrbi za avtentikacijo in avtorizacijo. Strežniški del je prav tako odgovoren za komunikacijo s podatkovno bazo, kjer se hranijo trajni podatki aplikacije [fielding2000rest]. V sodobnih spletnih aplikacijah strežniški del pogosto omogoča dostop do podatkov preko programskih vmesnikov (vmesnik API). Eden izmed najpogostejše uporabljenih arhitekturnih pristopov je REST, ki temelji na brezstanovnem (angl. *stateless*) načinu komunikacije med sistemi in uporablja standardne HTTP metode za izvajanje operacij nad viri [fielding2000rest]. Poleg osnovne funkcionalnosti lahko strežniški del vključuje tudi dodatne mehanizme, kot so omejevanje števila zahtevkov (angl. *rate limiting*), predpomnjenje podatkov ter beleženje dogodkov, kar prispeva k večji varnosti, stabilnosti in zmogljivosti sistema [nginx_rate_limiting].



Slika 3.1: Arhitektura spletne aplikacije tipa odjemalec–strežnik.

3.2 Relacijske podatkovne baze

Relacijske podatkovne baze predstavljajo enega izmed najpogostejše uporabljenih pristopov za shranjevanje podatkov v spletnih aplikacijah. Temeljijo na relacijskem modelu, pri katerem so podatki organizirani v tabele, ki predstavljajo posamezne entitete. Vsaka entiteta vsebuje attribute, s katerimi opisujemo njene lastnosti, posamezne vrstice pa predstavljajo konkretne primerke entitete. Relacijski model omogoča strukturirano in konsistentno shranjevanje podatkov ter učinkovito izvajanje poizvedb nad velikimi količinami

podatkov [1]. Povezave med entitetami vzpostavimo s primarnimi in tujimi ključi. Primarni ključ enolično identificira posamezen zapis v tabeli, medtem ko tuji ključ omogoča povezovanje podatkov med različnimi tabelami. Na ta način zmanjšamo podvajanje podatkov ter zagotovimo referenčno integriteto podatkovne baze [1].

Pri razvoju sistema smo izbrali PostgreSQL, ki predstavlja odprtokodni objektno-relacijski sistem za upravljanje podatkovnih baz. PostgreSQL temelji na standardu SQL in ga razširja z naprednimi funkcionalnostmi, ki omogočajo učinkovito delo tako s strukturiranimi kot tudi s polstrukturiranimi podatki [5]. Med pomembnejšimi lastnostmi sistema PostgreSQL so podpora za transakcije ACID, napredne poizvedbe SQL, replikacija podatkov, mehanizmi za zagotavljanje integritete podatkov ter podpora za podatkovni format JSON oziroma JSONB. Zaradi teh funkcionalnosti je PostgreSQL primeren za razvoj sodobnih spletnih aplikacij, ki zahtevajo visoko zmogljivost, zanesljivost in možnost nadaljnje razširljivosti sistema [5, 4].

3.3 Evalvacijski sistem

Vrednotenje vozil na podlagi primerljivih vozil

Zajem in normalizacija podatkov

3.4 Mikrostoritvena arhitektura

Mikrostoritvena arhitektura je arhitekturni vzorec, pri katerem se programski sistem razdeli na množico manjših, med seboj neodvisnih storitev, namesto da bi bil zgrajen kot ena sama, tesno povezana enota, znana kot monolitna arhitektura [2]. Vsaka mikrostoritev implementira natančno določeno poslovno funkcionalnost in je razvita, nameščena ter skalirana neodvisno od preostalih storitev v sistemu. Komunikacija med posameznimi mikrostoritvami praviloma poteka prek dobro definiranih vmesnikov, kot so vmesniki

API, sporočilne vrste ali drugi mehanizmi za izmenjavo podatkov [3].

Razdelitev sistema na več manjših, neodvisnih komponent prinaša več prednosti v primerjavi z monolitno arhitekturo [3]. Posamezne mikrostoritve je lažje testirati, saj imajo omejen in jasno določen obseg delovanja. Prav tako je olajšano odkrivanje in odpravljanje napak, saj je napako mogoče hitreje lokalizirati znotraj posamezne storitve, namesto da bi jo iskali znotraj celotnega sistema. Neodvisnost storitev dodatno omogoča lažje vzdrževanje, saj je mogoče posodobiti ali zamenjati eno mikrostoritev, ne da bi to vplivalo na delovanje ostalih mikrostoritev v sistemu, pod pogojem, da vmesnik za komunikacijo med njimi ostane nespremenjen [2].

Poglavje 4

Struktura diplomske naloge

Diplomska naloga ima sicer predvideno okvirno strukturo, dejanska struktura pa je precej odvisna od področja v katerem delujete, zato se glede tega nujno posvetujte z mentorjem. Ustaljena struktura sicer bralcu, predvsem pa mentorju in članom komisije, omogoča hitro navigacijo in preverjanje znanstvene integritete opravljenega dela.

Tipična struktura diplomske naloge na FRI obsega naslednje obvezne in priporočene gradnike:

Naslovna stran poleg naslova v slovenščini in angleščini vsebuje še ime avtorja, mentorja, morebitnega somentorja ter letnico in kraj izdaje. Naslov mora biti jedrnat, a hkrati dovolj informativen, da bralec takoj prepozna domeno računalništva, ki jo naloga obravnava.

Izjava o avtorstvu je formalni dokument, s katerim študent jamči, da je diplomsko delo plod lastnega dela in da so vsi uporabljeni viri ustrezno navedeni.

- **Metodologija oz. teoretične osnove:** Razlaga konceptov, potrebnih za razumevanje rešitve.
- **Analiza in načrtovanje:** Opis arhitekture rešitve, omejitev, zahtev ipd.

- **Implementacija:** Opis rešitve, podrobnosti razvoja, uporabljene knjižnice in reševanje specifičnih tehničnih izzivov, vendar brez dejanskih števil.
- **Eksperimenti, evalvacija in rezultati:** Testiranje rešitve, primerjava zmogljivosti ali uporabniška študija, predstavitev rezultatov in argumentacija. Tipično se to razbije na več poglavij, odvisno od količine izvedenih eksperimentov.

Zaključek povzame dosežene cilje in poda kritično presojo opravljenega dela. Pomembno je, da izpostavite omejitve svoje rešitve in nakažete smeri za morebitno nadaljnje delo ali izboljšave.

Literatura in viri je seznam vseh strokovnih in znanstvenih člankov, knjig in spletnih virov, na katere se sklicujete. Vsak vir v seznamu mora imeti ustrezen sklic v besedilu.

Priloge so namenjene gradivu, ki bi preveč motilo tok besedila v osrednjem delu, a je pomembno za popolno razumevanje (npr. obsežni kosi programske kode, vprašalniki ali podrobni diagrami).

4.1 Slogovne smernice in priporočila

Zavedajte se, da vrstni red branja ne sledi vrstnemu redu pisanja zaključnega dela. Verjetno se boste lotili pisanja sorodnih del in metodologije pred eksperimenti ali rezultati, vsekakor pa pred uvodnim razdelkom ali povzetkom, ki ga zagotovo napišite povsem na koncu. Tega se zavedajo tudi bralci, zato se vam pri pisanju uvoda ni potrebno pretvarjati, da eksperimenti še niso bili opravljeni.

- Strokovna besedila morajo biti nedvoumna, zato naj bodo stavki kratki, preprosti in razumljivi, terminologija dosledna.
- Pri pisanju zaključnega dela upoštevajte akademski slog, ki se močno razlikuje od poljudnega pisanja. Besedilo pišite v prvi osebi množine

(npr. „razvili smo“, „opazimo“), saj s tem poudarite objektivnost raziskovalnega procesa, v katerega sta vključena študent in mentor. Dovoljena je tudi uporaba trpnika, ki pa je jezikovno okornejša („razvit je bil“, „izdelano je bilo“). Prve osebe ednine („razvil sem“, „izdelal sem“) ne uporabljajte oziroma jo uporabite izjemoma samo tam, kjer želite poudariti svojo osebno odločitev med več možnostmi.

- Vse trditve v nalogi morajo biti preverljive. To pomeni, da morajo temeljiti na lastnih izpeljavah ali testiranjih, če pa povzimate ugotovitve drugih, je navajanje vira obvezno.
- Za posamezen pojem ali objekt uporabljajte v celotnem delu enako ime in oznako (če je potrebna).
- Če vpeljujete nove pojme, pazite, da so nedvoumno in natančno definirani.
- Uporabljajte uveljavljeno strokovno terminologijo.
- S pisanjem strokovnih besedil se razvija tudi slovenska strokovna terminologija. Pri strokovnih izrazih, ki še nimajo prevoda v slovenščino, po potrebi in v sodelovanju z mentorjem poiščite ustrezen slovenski prevod.
- Posebna pozornost naj velja prevodom iz angleščine. Pazite na slovenska slovnična pravila. Samostalniški prilastki so v slovenščini na desni strani osebka ali predmeta (SQL query → poizvedba SQL, API interface → vmesnik API, x -axis → os x), obratni vrstni red, kot je na primer SQL poizvedba, Windows strežnik ali ACME sistem, je napačen.

Poglavje 5

Osnovni gradniki L^AT_EXa in nasveti za pisanje

L^AT_EX bi lahko najbolj preprosto opisali kot programski jezik namenjen oblikovanju besedil. Tako kot vsak visokonivojski programski jezik ima tudi L^AT_EX številne ukaze za oblikovanje besedila in okolja, ki omogočajo strukturiranje besedila.

Vsi L^AT_EXovi ukazi se začnejo z levo poševnico `\`, okolja pa definiramo bodisi s parom zavitih oklepajev `{ in }` ali z ukazoma `\begin{ }` in `\end{ }`. Ukazi imajo lahko tudi argumente, obvezni argumenti so podani v zavutih oklepajih, opcijski argumenti pa v oglatih oklepajih.

Z ukazi torej definiramo naslov in imena avtorjev besedila, poglavja in podpoglavja in po potrebi bolj podrobno strukturiramo besedila na spiske, navedke itd. Posebna okolja so namenjena zapisu matematičnih izrazov, kratki primeri so v naslednjem poglavju.

Vse besedilne konstrukte lahko poimenujemo in se s pomočjo teh imen nato kjerkoli v besedilu na njih tudi sklicujemo.

L^AT_EX sam razporeja besede v odstavke tako, da optimizira razmike med besedami v celotnem odstavku. Nov odstavek začnemo tako, da izpustimo v izvirnem besedilu prazno vrstico. Da besedilo skoči v novo vrstico pa ukažemo z dvema levima poševnicama. Število presledkov med besedami v

izvornem besedilo ni pomembno.

5.1 Koristni nasveti pri pisanju v L^AT_EXu

Programski paket L^AT_EX je bil prvotno predstavljen v priročniku in je v resnici nadgradnja sistema T_EX avtorja Donalda Knutha znanega po svojih knjigah o umetnosti programiranja ter Knuth-Bendixovem algoritmu T_EX in njegove izpeljanke so odprtokodni programi.

Različnih implementacij L^AT_EXa je cela vrsta, a priporočamo spletno verzijo, ki poenostavi sodelovanje pri pisanju, Overleaf.

Včasih smo si pri pisanju v L^AT_EXu pomagali predvsem s tiskanimi priročniki danes pa je enostavneje in hitreje, da ob vsakem problemu za pomoč enostavno povprašamo najljubši jezikovni model.

L^AT_EX včasih ne zna pravilno deliti slovenskih besed, ki vsebujejo črke s strešicami. Če taka beseda štrli preko desnega roba, lahko L^AT_EXu pokažemo, kje se tako besedo deli takole: `ra\-\ču\-\na\-\ni\-\štvo`. Katere vrstice so predolge lahko vidimo tako, da dokument prevedemo s vključeno opcijo `draft: \documentclass[a4paper, 12pt, draft]{book}`.

V izvornem besedilu lahko začnete vsak stavek v novi vrstici, saj L^AT_EX sam razporeja besede po vrsticah postavljenega besedila. Na ta način je tudi iskanje po izvornem besedilu in popravljanje veliko hitrejše. Večina sistemov za T_EXiranje sicer omogoča s klikanjem enostavno prestopanje iz prevedenega besedila na ustrezno mesto v izvornem besedilu in obratno.

Boljšo preglednost dosežemo tako kot pri pisanju programske kode – z vizualnim urejanjem kode in izpuščanjem praznih vrstic. Pri spreminjanju in dodajanju izvirnega besedila je najbolje pogosto prevajati, da se sproti prepričamo, če so naši nameni pravilno izpolnjeni.

Kadar besedilo, ki je že bilo napisano z nekim vizualnim urejevalnikom (npr. z Wordom), želimo prenesti v L^AT_EX, je tudi najbolje to delati postopoma s posameznimi bloki besedila, tako da lahko morebitne napake hitro identificiramo in odpravimo. Za prevajanje Wordovih datotek v L^AT_EX – in

obratno – sicer obstajajo prevajalniki, ki pa običajno ne generirajo tako čisto logične strukture besedila, kot jo sicer $\text{\LaTeX}$ omogoča. Hiter in enostaven način prevedbe besedila, ki zahteva sicer ročne dopolnitve, lahko poteka tudi tako, da besedilo urejeno z vizualnim urejevalnikom najprej shranimo v formatu pdf, nato pa to besedilo uvozimo v urejevalnik, kjer urejamo izvirno besedilo v formatu $\text{\LaTeX}$.

5.2 Pisave v $\text{\LaTeXu}$

V $\text{\LaTeX}$ ovem okolju lahko načeloma uporabljamo poljubne pisave. Izbira poljubne pisave pa ni tako enostavna kot v vizualnih urejevalnikih besedil. Posamezne oblikovno medseboj usklajene pisave so običajno združene v družine pisav. V $\text{\LaTeXu}$ se privzeta družina pisav imenuje Computer Modern, kjer so poleg navadnih črk (roman v $\text{\LaTeXu}$) na voljo tudi kurzivne črke (*italic* v $\text{\LaTeXu}$), krepke (**bold** v $\text{\LaTeXu}$), kapitelke (SMALL CAPS v $\text{\LaTeXu}$), linearne črke (*san serif* v $\text{\LaTeXu}$), pisava pisalnega stroja (*typewriter* v $\text{\LaTeXu}$) in nekatere njihove kombinacije, npr. krepke linearne črke (**san serif** v $\text{\LaTeXu}$). V istem dokumentu zaradi skladnega izleda uporabljamo običajno le pisave ene družine. Pomembna je tudi konsistentna raba večih pisav in da ne pretiravamo z mešanjem različnih pisav.

Ko začnemo uporabljati $\text{\LaTeX}$, je zato najbolj smiselno uporabljati kar privzete pisave, s katerimi je napisan tudi ta dokument. Z ustreznimi ukazi lahko nato preklapljammo med navadnimi, kurzivnimi, krepkimi in drugimi pisavami. Zelo enostavna je tudi izbira velikosti črk. $\text{\LaTeX}$ odlično podpira večjezičnost, tudi v sklopu istega dokumenta, saj obstajajo pisave za praktično vse jezike, tudi take, ki ne uporabljajo latinskih črk.

5.3 Skladnost s standardom PDF/A

Repozitorij Univerze v Ljubljani zahteva, da so oddana zaključna dela skladna s standardom PDF/A. V predlogi smo zagotovili skladnost s standardom

PDF/A-2b. S predlogo žal ne moremo preprečiti, da bi uporabnik v diplomsko delo dodal elemente, ki niso skladni s standardom. Pred oddajo zaključnega dela zato uporabnikom priporočamo preverjanje skladnosti z ustreznimi preverjevalniki, npr. z orodjem veraPDF¹.

5.4 Uporaba količin in enot

Pri tehničnem in strokovnem pisanju je zelo pomembno dosledno razlikovanje med količinami in njihovimi pripadajočimi merskimi enotami. Neustrezen zapis lahko vodi do napačne interpretacije podatkov in zmanjša strokovno vrednost dokumenta.

Količina predstavlja lastnost sistema, strojne ali programske opreme, ki jo je mogoče kvantitativno določiti. Simbole količin (npr. C za kapaciteto pomnilnika, f za frekvenco procesorja, B za pasovno širino) pišemo praviloma v **ležeči pisavi**.

Merska enota je dogovorjena vrednost, s katero primerjamo isto vrsto količine. Simbole enot (npr. B, Hz, bit s^{-1}) pišemo v **pokončni pisavi**.

Med številčno vrednostjo in pripadajočo enoto mora biti vedno prisoten **stični presledek** (tilda), tako kot pri citatih. To zagotavlja, da vrednost in enota ob morebitnem prelomu vrstice ostaneta skupaj.

Za dosego standardiziranega zapisa v okolju L^AT_EX uporabljamo paket `siunitx`. Ta omogoča avtomatsko upravljanje s presledki in decimalnimi ločili. Spodaj so navedeni primeri pravilne uporabe:

- **Izpis vrednosti z enoto:** Ukaz `\qty{100}{\giga\byte}` vrne 100 GB.
- **Posebni znaki:** Odstotke in stopinje pišemo s presledkom (za to že poskrbi L^AT_EX), npr.:

– `\qty{98}{\percent}` vrne 98 %,

¹<https://verapdf.org/>

– `\qty{85}{\degreeCelsius}` vrne 85°C .

V slovenščini kot decimalno ločilo uporabljamo vejico, kar v paketu `siunitx` nastavimo globalno z ukazom `output-decimal-marker = {,}`.

Še primer rabe enot v besedilu: Pri obremenitvenem testu strežnika smo ugotovili, da pri frekvenci $f = 3,5\text{ GHz}$ in obremenitvi 98% prepustnost omrežja doseže $B = 1,2\text{ Gbit s}^{-1}$. Vse meritve so bile opravljene pri temperaturi procesorja $T = 85^{\circ}\text{C} \pm 1,5^{\circ}\text{C}$.

5.4.1 Računalnikarji, spoštujte SI!

Pri pisanju se zavedajte, da cel svet, z izjemo Liberije, Mjanmarja in Združenih držav Amerike, uradno uporablja Mednarodni sistem enot (SI), ki nam vsakodnevno lajša življenje. Ali ni čudovito reči, da ste letos prevozili 23 Mm , namesto nepoučenega dolgovezenja s $23\,000\text{ km}$?

Pri zapisovanju velikosti v bajtih prihaja zaradi zgodovinskih razlogov do pogostih težav. Velja sledeče:

- Če ste uporabili desetiški pretvornik (1000), potem uporabite SI predpone: k (kilo), M (mega), G (giga), T (tera), tj. kB, MB, GB, TB.
- Če ste uporabili dvojiški pretvornik (1024), potem uporabite dvojiške predpone: kibi (Ki), mebi (Mi), gibi (Gi), tebi (Ti), tj. KiB, MiB, GiB, TiB. Naj vas ne zmoti dejstvo, da se veliko proizvajalcev in celo določeni operacijski sistemi tega ne držijo (lahko da prihajajo iz katere od treh posebnih držav).

Seveda ne pozabite tudi na razliko v zapisu bajta B in bita bit!

5.5 Sklici in citiranje

Pri pisanju strokovnih besedil je nujno dosledno sklicevanje na slike, tabele, enačbe in druga poglavja ter citiranje uporabljene literature. Sistem $\text{\LaTeX}$

nam to močno olajša, saj ob prevajanju samodejno poskrbi za pravilno oštevilčenje. Uporabljamo naslednje osnovne ukaze:

- **Označevanje** (`\label`): Element, na katerega se želimo sklicevati, moramo najprej označiti z ukazom `\label{ime_oznake}`. Priporočljivo je, da za boljšo preglednost uporabljate predpone, na primer `sl`: za slike, `tab`: za tabele, `en`: za enačbe in `pogl`: za poglavja (npr. `\label{sl:galaksija}` ali `\label{ch:osnove}`).
- **Sklicevanje** (`\ref`): V besedilu se na označeni element skličemo z ukazom `\ref{ime_oznake}`. $\text{\LaTeX}$ bo na tem mestu izpisal ustrezno številko. Če se sklicujemo na matematično enačbo, namesto tega uporabimo ukaz `\eqref{ime_oznake}`, ki številko enačbe samodejno obda z okroglimi oklepaji.
- **Citiranje** (`\cite`): Za navajanje virov iz bibliografske datoteke uporabimo ukaz.

Zelo pomembno pravilo: Med besedo in samim sklicem ali citatom mora vedno stati **nedeljivi presledek**, ki ga v $\text{\LaTeX}$ u označimo s tilde (`~`). S tem preprečimo, da bi številka sklica ob prelomu vrstice padla sama v novo vrstico, kar je tipografsko nepravilno.

Pravilna raba v besedilu torej izgleda takole: Kot lahko vidimo na sliki `\ref{sl:galaksija}`, velja enačba `\eqref{en:masa}`. Podobno velja za literaturo: Algoritem je opisan v poglavju

Poglavje 6

Pogoste napake pri pisanju v slovenščini

Pri pisanju strokovnih besedil v slovenščini se pogosto srečujemo s tipičnimi napakami, ki izvirajo iz vpliva tujih jezikov ali nepoznavanja tipografskih pravil L^AT_EXu. Spodaj so zbrana ključna pravila, organizirana po kategorijah:

- **Vrstni red besed in kratic:** Pridevniki, ki se ne sklanjajo (npr. *kratice*), morajo v slovenščini stati za samostalnikom. Pravilno pišemo „model CAD“ in **ne** „CAD model“.
- **Sklanjanje tujih imen:** Pri sklanjanju tujih lastnih imen v slovenščini ne uporabljamo vezajev. Pravilno je „Applov operacijski sistem“ in **ne** „Apple-ov“.
- **Ločila in stičnost:** Pika, klicaj in vprašaj so levostični (pred njimi ni presledka, za njimi pa je). Klicajev in vprašajev se v strokovnih besedilih načeloma izogibamo. Oklepaji so desnostični, zaklepaji pa levostični: „(takole)“.
- **Tropičje in naštevanje:** Tropičja enostavno v diplomah ne uporabljajte. V znanstvenih besedilih in diplomah uporabljamo *itd.* ali *idr.* in ne *tropičja*. Pozorni bodite, da lahko uporabite le *npr.* ali *itd.* in ne obeh v eni povedi.

Za kasnejšo rabo tropičja v življenju pa vendarle velja sledeče. V slovenščini je tropičje praviloma nestično ločilo. V $\text{\LaTeX}$ u ga zapišemo z ukazom `\dots` in ne s tremi pikami. Primer: „pes, maček, rosomah, ...“ Izjema je tropičje, ki mu sledi drugo ločilo; takrat je levonestično, npr. „pes, maček, rosomah, ...!“

- **Narekovaji:** So stična ločila. V $\text{\LaTeX}$ u ne smemo uporabiti zgolj tipke " ali dveh ’’, saj se začetni in končni narekovaji ne bodo pravilno izrisali. Najboljša praksa je uporaba ukaza `\enquote{...}`, ki samodejno postavi pravilne slovenske narekovaje: „spodaj devetice, zgoraj šestice“.
- **Vežaj in pomišljaj:**
 - **Vežaj (-):** Je levo in desno stičen, npr. **slovensko-angleški** in označuje enakovredno razmerje med pojmomoma. V kodi ga pišemo z enim znakom `-`.
 - **Pomišljaj (–):** V slovenščini in v (britanski) angleščini je nestičen (s presledki). V kodi ga zapišemo z dvema znakoma `-`, npr. **Pozor -- hud pes!**
 - **Razponski pomišljaj (—):** Uporabljamo ga za razpone (npr. „od–do“), je stičen: **strani 7--11** in se ga v kodi piše povsem enako kot pomišljaj `--`.
 - **Ameriški pomišljaj (—):** t.i. *em-dash* je daljši, stičen in se v kodi piše s tremi znaki `---`, opravlja pa povsem enako vlogo kot nestični pomišljaj v slovenščini oz. v britanski angleščini t.i. *en-dash*, npr. **Beware---vicious dog!** V slovenščini ga seveda ne uporabljajte.
- **Presledki ob ločilih in kraticah:** $\text{\LaTeX}$ privzeto uporablja t.i. francoski razmik, kar pomeni, da je presledek po koncu stavka malenkost večji od tistega med besedami. To vodi do dveh specifičnih težav:

- **Kratice sredi stavka:** Če piki za kratico (npr. *prof.*, *dr.*, *itd.*) sledi presledek, bo L^AT_EX to interpretiral kot konec stavka in vstavil predolg razmik. To preprečimo z uporabo tilde ~ (stični presledek) ali poševnice s presledkom \ . Pravilno: `prof.~dr.~Novak` ali `npr.\ v tem primeru.`
- **Kratice na koncu stavka:** Če se stavek konča z veliko črko (npr. kratica na koncu stavka), L^AT_EX misli, da gre za kratico sredi stavka in vstavi premajhen razmik. Za pravilno interpretacijo konca stavka pred piko vstavimo ukaz \@. Pravilno: `Razvili so novo CPE\@.` Sledila je testna faza.
- **Vejice in stavčna zgradba:** Vejica loči stavke (povedke), razen če so zvezani z vezniki *in*, *pa*, *ter*, *ne-ne*, *nití-nití*, *ali*, *bodisi*, *oziroma*. Čeprav velja pravilo „pred ki, ko, ker, da, če, vejica skače“, je potrebno paziti na vrivke in večbesedne veznike. V strokovnih besedilih je bolje pisati krajše in jasne stavke. Veliki jezikovni modeli vam bodo pri postavljanju vejic v veliko pomoč, za zares pravilno postavljanje pa svetujemo uporabo spletnega orodja CJVT vejice (<https://orodja.cjvt.si/vejice/>).
- **Terminologija in viri:** Za nove računalniške pojme si pomagamo z uveljavljenimi slovarji. Glavni viri so *Islovar* Slovenskega društva InformatikaSlovar Slovenskega društva za razpoznavanje vzorcev ter osrednje jezikovno spletišče *Fran*

Poglavje 7

Plovke: slike in tabele

Slike in daljše tabele praviloma vključujemo v dokument kot plovke. Pozicija plovke v končnem izdelku ni pogojena s tekom besedila, temveč z izgledom strani. L^AT_EX bo skušal plovko postaviti samostojno, praviloma na mestu, kjer se pojavi v izvornem besedilu, sicer pa na vrhu strani, na kateri se na takšno plovko prvič sklicujemo. Pri tem pa bo na vsako stran končnega izdelka želel postaviti tudi sorazmerno velik del besedila. V skrajnem primeru, če imamo res preveč plovk na enem mestu besedila, ali če je plovka previsoka, se bo L^AT_EX odločil za stran popolnoma zapolnjeno s plovkami.

Poleg tega, da na položaj plovke vplivamo s tem, kam jo umestimo v izvorno besedilo, lahko na položaj plovke na posamezni strani prevedenega besedila dodatno vplivamo z opcijami `here`, `top` in `bottom`. Zelo velike slike je najbolje postaviti na posebno stran z opcijo `page`. Skaliranje slik po njihovi širini lahko prilagodimo širini strani tako, da kot enoto za širino uporabimo kar širino strani, npr. `0.5\textwidth` bo raztegnilo sliko na polovico širine strani. Sliko lahko po potrebi tudi zavrtimo za 90 stopinj in jo razstegnemo na višino strani. Tako bodo podrobnosti na sliki lažje berljive in prostor na strani bo bolje izkoriščen.

Na vse plovke se moramo v besedilu sklicevati, saj kot beseda plovka pove, plovke plovejo po besedilu in se ne pojavijo točno tam, kjer nastopajo v izvornem besedilu. Vendar naj bosta sklic na plovko v besedilu in sama plovka v

oblikovanem besedilu čim bližje skupaj, tako da bralcu ne bo potrebno listati po diplomih. Upoštevajte pa, da se naloge tiska dvostransko in da se hkrati vidi dve strani v dokumentu! Na to, kje se bo slika ali druga plovka pojavila v postavljenem besedilu lahko torej najbolj vplivamo tako, da v izvorni kodi plovko premikamo po besedilu nazaj ali naprej!

Tabele je najbolje oblikovati kar neposredno v $\text{\LaTeX}$ u, saj za oblikovanje tabel obstaja zelo fleksibilno okolje `tabular`. Slike pa je po drugi strani pogosto najlažje oblikovati oziroma izdelati z drugimi orodji in programi, rezultate shraniti v formatu `.pdf` ali `.jpeg` in nato v $\text{\LaTeX}$ u le vključiti ustrezno slikovno datoteko. Za pisanje besed, ki so vključene v slike, uporabite pisave/fonte, ki so čim bolj podobne pisavam v samem besedilu, ali še bolje shranite brez fonta in pustite $\text{\LaTeX}$ u, da vključi svojega.

Knjižnica <https://en.wikibooks.org/wiki/LaTeX/PGF/TikZ> pa omogoča risanje raznovrstnih grafov neposredno v okolju $\text{\LaTeX}$.

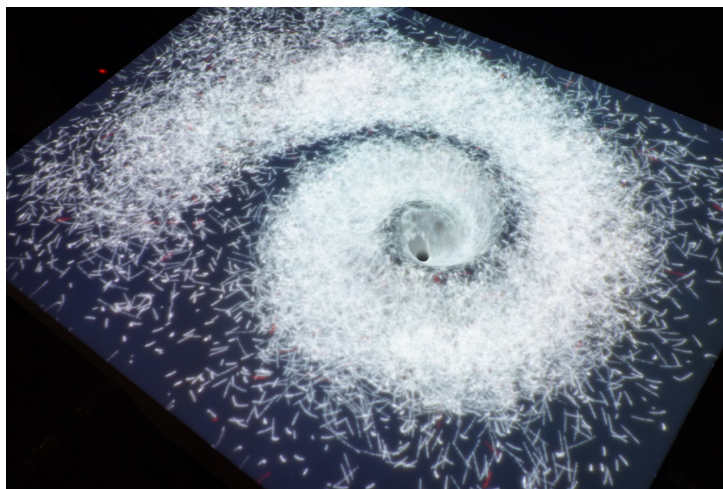
Na vse tabele in slike se moramo v besedilu sklicevati, saj kot plovke v oblikovanem besedilu niso nujno na istem mestu kot v izvornem besedilu. Pri sklicevanju na slike uporabimo veliko začetnico, npr. "glej Sliko ", saj gre za ime slike.

7.1 Formati slik

V dokument $\text{\LaTeX}$ lahko vključimo slike različnih formatov, tako rasterske slike kot vektorske slike.

Za grafe in diagrame so najbolj primerne vektorske slike v formatu `.pdf`. Ne samo, da bo besedilo predstavljeno kot besedilo, ampak bodo slike lepe in berljive tudi ob veliki povečavi. Še več, shranjevanje v vektorski obliki od vas običajno ne zahteva nič dodatnega truda. Če npr. graf narišete v Pythonu, le poskrbite, da ga shranite kot `.pdf` in ne kot npr. `.png`, nikakor pa ne kot `.jpg`.

Če vključujete fotografije oz. nekaj česar niste narisali pa bo to verjetno shranjeno v rasterski obliki in shranjeno kot `.jpg` ali `.png`. Slika je npr. v



Slika 7.1: Virtualno obogatena skulptura. Rezultate računalniško generirane animacije z video projektorjem projeciramo na kamnito skulpturo, da ustvarimo vtis, kot da bi po skulpturi polzele vodne kapljice

formatu .jpg, vendar na povsem enak način vključite tudi npr. pdf.

7.2 Podnapisi k slikam in tabelam

Vsaki sliki ali tabeli moramo dodati podnapis, ki na pojasnjuje, kaj je na sliki ali tabeli, obrazloži vse morebitne okrajšave, pojasni glavna dognanja. Ne bojte se daljših opisov. Če nekdo le prelista diplomsko delo, naj bi že iz slik in njihovih podnapisov lahko na grobo razbral, kakšno temo naloga obravnava.

Če slike povzamemo iz drugih virov, se moramo v podnapisu k taki sliki nujno sklicevati na ta vir!

7.3 Vključevanje več slik in tabel hkrati

Pri pisanju diplomske naloge se pogosto pojavi potreba po prikazovanju več slik ali tabel ene ob drugi. Za ta namen uporabljamo paket `subcaption`, ki

predstavlja sodoben in podprt standard v L^AT_EXu.

V spletnih virih in starih predlogah študenti prepogosto najdejo in uporabijo paketa `subfigure` ali `subfig`. Oba paketa sta zastarela in pogosto povzročata težave pri prevajanju ali konflikte z drugimi paketi (npr. pri generiranju hiperpovezav). Teh dveh paketov zato **ne uporabljajte**. Prav tako se za razporejanje slik v mrežo ne poslužujte paketa `TikZ`. Ta paket je sicer izjemno močno orodje, a je namenjen risanju vektorskih shem in je za preprosto razporejanje slik odločno preveč kompleksen.

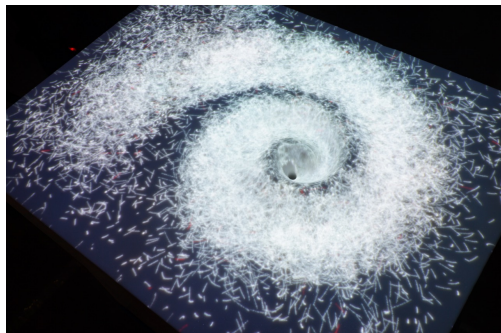
Za uporabo sodobnega pristopa v preambulo dokumenta dodajte ukaz `\usepackage{subcaption}`. Paket nam ponuja dve okolji: `subfigure` za slike in `subtable` za tabele. Obe okolji v ozadju delujeta na podobnem principu kot standardno okolje `minipage` (ki ga v L^AT_EXu pogosto uporabljamo za postavljanje povsem neodvisnih blokov vsebin enega ob drugega), saj obema podamo eksplicitno širino, ki jo bosta zavzela.

Ker so slike in tabele plavajoči objekti, ne smemo nikoli predpostavljati, kje natančno na strani se bodo izrisale. Na vse elemente se v besedilu vedno sklicujemo izključno z ukazom `\ref`, pred katerim uporabimo nedeljivi presledek (`~`).

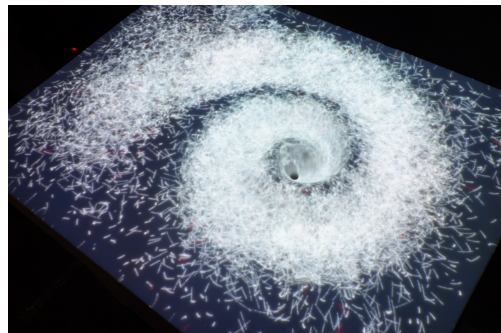
7.3.1 Več slik s podnaslovi

Kadar želimo vsaki sliki dodeliti svojo oznako (npr. a, b) in podnaslov, uporabimo okolje `subfigure`. Širino posamezne podslike določimo z ulomkom širine besedila, npr. `0.48\textwidth`. Vmesni prostor do polne širine zapolnimo z ukazom `\hfill`. Na sliki 7.2 lahko vidimo primer dveh slik, ki sta postavljeni ena ob drugi.

Za prehod v novo vrsto znotraj istega plavajočega objekta preprosto izpustimo prazno vrstico in po želji dodamo nekaj vertikalnega prostora z ukazom `\vspace`. Na tak način je zgrajena mreža štirih slik na sliki 7.3.

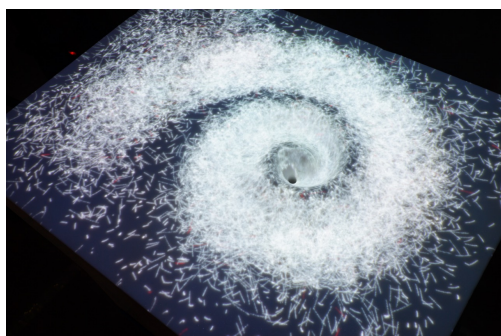


(a) Podnaslov leve slike.

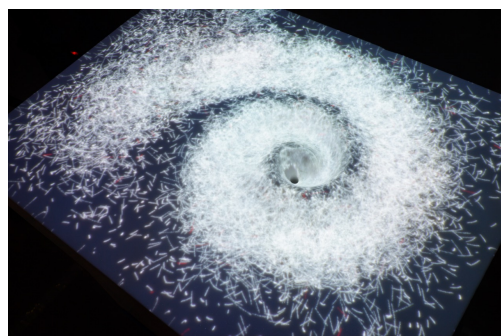


(b) Podnaslov desne slike.

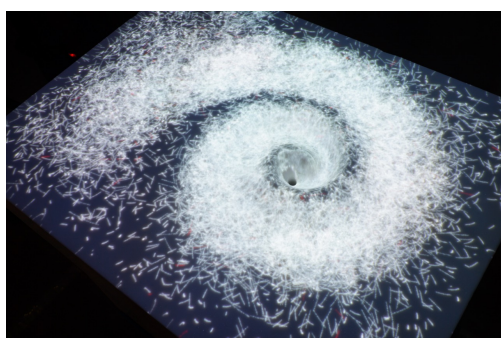
Slika 7.2: Skupni naslov, ki opisuje obe sliki.



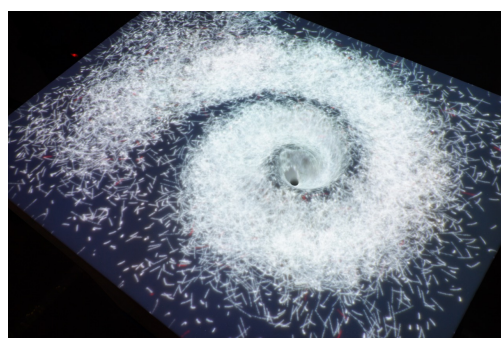
(a) Zgoraj levo.



(b) Zgoraj desno.



(c) Spodaj levo.



(d) Spodaj desno.

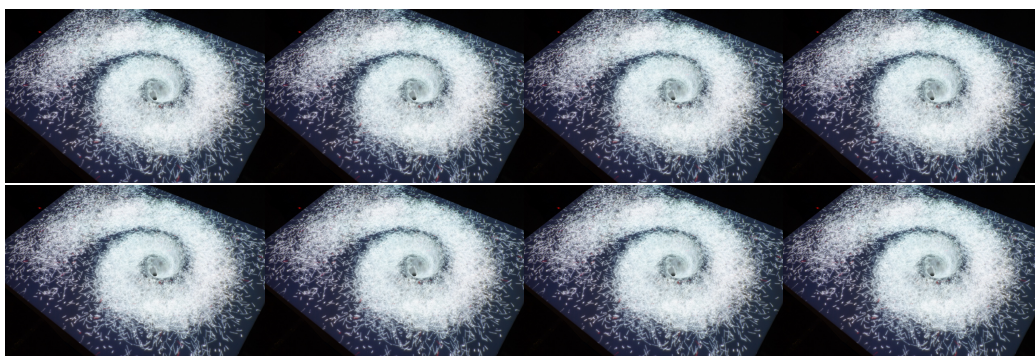
Slika 7.3: Mreža štirih slik s posameznimi podnaslovi.

7.3.2 Več slik brez podnaslovov (mreža brez presledkov)

Če individualni podnaslovi niso potrebni in želimo le en skupen naslov celotne mreže slik, okolja `subfigure` sploh ne potrebujemo. Slike preprosto naložimo znotraj navadnega `figure` okolja in določimo njihovo velikost.

Če želimo ustvariti neprekinjeno mrežo povsem **brez vmesnih presledkov** (kot to prikazuje slika 7.4), moramo poskrbeti za dve stvari:

1. **Horizontalni presledki:** Širino določimo tako, da je vsota vseh širin v vrstici točno enaka `1.0\textwidth` (npr. štiri slike s širino `0.25\textwidth`). Na konec vsake vrstice z ukazom `\includegraphics` moramo dodati znak za komentar (`%`), s čimer preprečimo, da bi $\text{\LaTeX}$ prelom vrstice interpretiral kot presledek.
2. **Vertikalni presledki:** Znotraj posebnega bloka moramo uporabiti ukaz `\offinterlineskip`, ki v celoti izklopi samodejno vrivanje vertikalnega prostora med vrsticami. Pomembno je, da blok po zadnji sliki zapremo z, preden definiramo `\caption`.



Slika 7.4: Mreža osmih slik brez vmesnih presledkov in z le enim skupnim naslovom.

7.3.3 Več tabel ena ob drugi

Logika postavljanja dveh tabel ene ob drugo je popolnoma enaka kot pri slikah, le da znotraj okolja `table` uporabimo okolje `subtable`. Edino po-

membno pravilo, na katerega ne smete pozabiti, je: **podnaslovi tabel vedno sodijo nad tabelo**, medtem ko podnaslovi slik sodijo pod njo.

Če ugotovite, da je vsebina tabele preširoka in povzroča prekrivanje, lahko celotno okolje `tabular` ovijete v ukaz `\resizebox{\linewidth}{!}{...}`, ki tabelo proporcionalno pomanjša, da se natančno prilega širini podtabele. Tabela 7.1 prikazuje ustrezen zapis na primeru vrednotenja modelov računalniškega vida.

Tabela 7.1: Primerjava uspešnosti klasifikacije slik z različnimi arhitekturami globokih nevronske mrež na dveh standardnih podatkovnih zbirkah. Pri metriki *točnost* (angl. *accuracy*) so boljše višje vrednosti (označeno s puščico $\uparrow$), pri metriki *napaka* (angl. *error rate*) pa so boljše nižje vrednosti (označeno s puščico $\downarrow$).

(a) Rezultati na zbirki CIFAR-10.

arhitektura	točnost ($\uparrow$)	napaka ($\downarrow$)
ResNet-18	93,5 %	6,5 %
ResNet-50	95,2 %	4,8 %
ViT-Base	97,8 %	2,2 %

(b) Rezultati na zbirki ImageNet.

arhitektura	točnost ($\uparrow$)	napaka ($\downarrow$)
ResNet-18	69,7 %	30,3 %
ResNet-50	76,1 %	23,9 %
ViT-Base	84,2 %	15,8 %

Poglavje 8

Vključevanje programske kode in algoritmov

Svojo programsko kodo v poljubnem programskem jeziku najlažje vključimo s paketom `minted`. Okoli bloka `\minted{ }` sicer lahko podamo klasični `\listing{ }` in skupaj z dodanim `\caption` dobimo vse potrebno za prikaz kode, vendar pa se takšen blok ne razteza pravilno skozi več strani. Ena od možnosti za rešitev je uporaba bloka `\center{ }`, za napis pa `\captionof{ }`, kot je prikazano pri obeh primerih izsekov kode 1 in 2.

Izsek kode 1: Kratek primer kode v programskem jeziku Python.

```
1 def python_veverica():  
2     # Oh, kako je to lepo!  
3     x = -1  
4     print("Res izgleda čudovito.")
```

Kar je nadvse čudovito pri paketu `minted` je to, da lahko podamo dele kode znotraj vrstice, kot je npr. tole: `python_veverica()`.

Izsek kode 2: Malo daljši primer programske kode v jeziku Javascript, ki se razteza skozi dve strani.

```
1 async function pridobiUporabnika(id) {  
2   const url = `https://api.primer.fri/users/${id}`;  
3  
4   try {  
5     const odgovor = await fetch(url);  
6  
7     // Preverimo, če je statusni kod v območju 200-299  
8     if (!odgovor.ok) {  
9       console.error(`Napaka pri povezavi: ${odgovor.status}`);  
10      throw new Error(`Uporabnika z ID ${id} ni bilo mogoče  
    ↪ najti.`);  
11    }  
12  
13    const podatki = await odgovor.json();  
14  
15    // Logiranje uspešnega prejema podatkov  
16    console.log("Podatki so bili uspešno prejeti.");  
17    console.log(`Ime uporabnika: ${podatki.name}`);  
18  
19    return podatki;  
20  
21  } catch (napaka) {  
22    // Tukaj ulovimo tako mrežne napake kot napake pri branju JSON-a  
23    console.warn("Prišlo je do nepričakovane težave.");  
24    return { error: napaka.message };  
25  }  
26 }
```

Vodoravni črti pri začetku in koncu izseka, v kombinaciji s sivim ozadjem, vizualno sporočata morebitno raztezanje skozi več strani. To je razvidno pri izseku kode 2.

8.1 Vključevanje algoritmov (psevdokode)

Medtem ko paket `minted` uporabljamo za prikaz dejanske implementacije v specifičnem programskem jeziku, v diplomskih nalogah pogosto opisujemo samo logiko reševanja problema na višjem nivoju. Za zapis algoritmov in obliki psevdokode je standardna izbira kombinacija paketov `algorithm` in `algpseudocode` (slednji je del paketa `algorithmicx`).

Za razliko od izsekov izvirne kode, algoritme običajno obravnavamo kot standardne plavajoče objekte (tako kot slike in tabele). To pomeni, da $\text{\LaTeX}$ sam poišče optimalno mesto za njihov prikaz na strani. V besedilu se nanje sklicujemo z nedeljivim presledkom in ukazom `\ref`, na primer: Algoritem 1 prikazuje klasičen postopek za iskanje največjega skupnega delitelja.

Naslov (`\caption`) pri algoritmih običajno postavljamo na vrh, tik pod začetek okolja `algorithm`. Številka [1] ob ukazu `\begin{algorithmic}` pa $\text{\LaTeX}$ u sporoči, naj oštevilči vsako vrstico algoritma, kar nam olajša sklicovanje na posamezne korake v besedilu.

Algorithm 1 Evklidov algoritem za iskanje največjega skupnega delitelja

Require: $a \geq 0 \vee b \geq 0$ **Ensure:** Največji skupni delitelj a in b

```
1:
2: procedure EVKLID( $a, b$ )
3:   while  $b \neq 0$  do
4:      $t \leftarrow b$ 
5:      $b \leftarrow a \bmod b$ 
6:      $a \leftarrow t$ 
7:   end while
8:   return  $a$ 
9: end procedure
```

Poglavje 9

Matematično okolje in sklicevanje na besedilne konstrukte

Poglejmo nekaj primerov pisanja enačb, sklicevanj nanje, podajanja dokazov, matrik, razbijanje enačb in nekaj splošnih tipografskih pravil v matematiki.

Izrek 9.1. *Za vsako naravno število n velja*

$$n < 2^n. \tag{9.1}$$

Dokaz. Dokazovanje z indukcijo zahteva, da neenakost (9.1) najprej preverimo za najmanjše naravno število – 0. Res, ker je $0 < 1 = 2^0$, je neenačba (9.1) za $n = 0$ izpolnjena.

Enačbo (9.1) smo referencirali z `\eqref{eq:1}` in ne z `\ref{eq:1}`, saj nam slednji ne ovije reference v oklepaje. Brez oklepajev bi bralec lahko referenco na enačbo zamenjal z referenco na poglavje, z oglatimi oklepaji pa na sklic v literaturi.

Za potrebe primera nadaljujmo z indukcijskim korakom. S predpostavko, da je neenakost (9.1) veljavna pri nekem naravnem številu n , je potrebno pokazati, da je ista neenakost v veljavi tudi pri njegovem nasledniku – naravnem

številu $n + 1$. Računajmo.

$$n + 1 < 2^n + 1 \quad (9.2)$$

$$\leq 2^n + 2^n \quad (9.3)$$

$$= 2^{n+1}$$

Neenakost (9.2) je posledica indukcijske predpostavke, neenakost (9.3) pa enostavno dejstvo, da je za vsako naravno število n izraz 2^n vsaj tako velik kot 1. S tem je dokaz Izreka 9.1 zaključen. $\square$

Ste opazili kako so indukcijski koraki lepo poravnani? Uporabili smo pomemben `align`. Ste opazili, da smo stavek pri neenakosti 9.1 končali s piko? V primeru, da je enačba del povedi, kot npr.

$$y = 2^x, \quad (9.4)$$

pa seveda dodamo vejico v samo enačbo, kot je prikazno pri (9.4).

Za kakovostno tehnično pisanje pa moramo obvladati še zapis matrik in obdelavo zelo dolgih enačb.

9.1 Matrike

Za zapis matrik uporabljamo okolja iz paketa `amsmath`. Najpogosteje uporabljamo okolje `bmatrix` za matrike z oglatimi oklepaji ali `pmatrix` za matrike z okroglimi oklepaji.

Primer zapisa matrike A :

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}. \quad (9.5)$$

Pri tem elemente v vrstici ločimo z znakom `&`, vrstice pa zaključimo z `\\`. Za različne vrste pik uporabljamo ukaze `\dots` (vodoravne), `\vdots` (navpične) in `\ddots` (diagonalne).

9.2 Razbijanje dolgih enačb

Kadar je enačba predolga za eno vrstico, uporabimo okolje `split` znotraj okolja `equation`. To nam omogoča, da enačbo razbijemo na več vrstic, medtem ko celoten blok obdrži le eno skupno številko.

$$\begin{aligned} f(x) &= (a + b)^4 \\ &= a^4 + 4a^3b + 6a^2b^2 + 4ab^3 + b^4. \end{aligned} \tag{9.6}$$

Znak `&` določa točko poravnave (običajno pri enačaju), `\\` pa prehod v novo vrstico.

Poglavje 10

Kaj pa literatura?

Kot smo omenili že v uvodu, je pravi način za citiranje literature uporaba `BIBLATEX`a. `BIBLATEX` zagotovi, da pri določeni vrsti literature ne izpustimo nobene obvezne informacije in da vse informacije dosledno navajamo na enak način in po istem vrstnem redu. `BIBLATEX` je nadgradnja starejšega sistema `BIBTEX`. Novejši sistem je bolje prilagojen slovenščini in navajanju spletnih virov. Sicer pa so starejše datoteke `.bib` kompatibilne z `BIBLATEX`om.

Osnovna ideja `BIBLATEX`a je, da vse informacije o literaturi zapisujemo v posebno datoteko, v našem primeru je to `literatura.bib`. Vsakemu viru v tej datoteki določimo simbolično ime. V našem primeru je v tej datoteki nekaj najbolj značilnih zvrsti literature, kot so knjige članki v revijah in zbornikih konferenc poglavja v knjigah spletni viri tehnično poročilo diplome itd. Diploma iz leta 1990 je bila prva diploma na tedanji Fakulteti za elektrotehniko in računalništvo, ki je bila oblikovana z `LATEX`om. Reference, ki so na spletnih straneh arhivirane v elektronski obliki, imajo običajno številko DOI (<http://dx.doi.org>), ki jo zato tudi vključimo v izpis literature in tako bralcu elektronske verzije naše publikacije ponudimo neposredno povezavo do elektronske kopije te reference.

Po vsaki spremembi pri sklicu na literaturo moramo najprej prevesti izvirno besedilo s prevajalnikom `LATEX`, nato s prevajalnikom `BIBLATEX`, ki ustvari datoteko `vzorec_dip_Seminar.bbl`, in nato še dvakrat s prevajalni-

kom $\text{\LaTeX}$. V okolju Overleaf je to večkratno prevajanje z različnimi prevajalniki uporabniku skrito. Zato tudi začetnim uporabnikom $\text{\LaTeX}$ a svetujemo uporabo Overleafa.

Kako se spisek literature nato izpiše (ali so posamezni viri razvrščeni po vrstnem redu sklicevanja, ali po abecedi priimkov prvih avtorjev, ali se imena avtorjev pišejo pred priimki itd.) je odvisno od parametrov paketa $\text{BIB}\text{\LaTeX}$. V diplomu bomo uporabili parametre `style=numeric`, kar pomeni, da bodo sklici na literaturo v besedilu označeni z zaporednimi številkami, za vrstni red izpisa referenc pa `sorting=nty`, kar pomeni, da bodo reference urejene po priimkih prvih avtorjev, nato po naslovu reference in nazadnje po letu izdaje. Zato je potrebno pri določenih zvrsteh literature, ki nima avtorjev, dodati parameter `key`, ki določi vrstni red vira po abecedi.

Ko začnemo uporabljati $\text{BIB}\text{\LaTeX}$ je lažje, če za urejanje datoteke `.bib` uporabljamo kar isti urejevalnik kot za urejanje datotek `.tex`, čeprav obstajajo tudi posebni urejevalniki oziroma programi za delo z datotekami `.bib`.

Le če se na določen vir v besedilu tudi sklicujemo, se bo ta vir pojavil tudi v spisku literature. Tako je avtomatično zagotovljeno, da se na vsak vir v seznamu literature tudi sklicujemo v besedilu diplome. V datoteki `.bib` imamo sicer lahko veliko več virov za literaturo, kot jih bomo uporabili v diplomu.

Poglavje 11

Sklepne ugotovitve

Uporaba L^AT_EXa in BIBL^AT_EXa je pri diplomski nalogi **obvezna**!

Preden diplomo oddate mentorjem, še enkrat preverite:

- pravilno rabo elementov (tako L^AT_EXgradnikov kot tudi ločil kratic, pomišljajev itd.),
- slovnico (lahko si pomagate s katerim od orodij),
- da ni nobeno besedilo prilepljeno direktno iz velikega jezikovnega modela (čisto vse morate prebrati in preveriti, če že ne popraviti),
- da je dovolj slikovnih elementov (preveč suhoparnega besedila ni za nikogar koristno),
- da je vsaka slika in tabela ne samo podnaslovljena, ampak tudi referencirana v besedilu.

Preden diplomo oddate na sistemu STUDIS, še enkrat preverite:

- če je izpisana pravilna smer študija, vaše ime, ime in naziv mentorja in somentorja,
- če so vse besede pravilno deljene in da ne segajo preko desnega roba (poravnava po vrsticah lahko kontrolirate tako, da izvirno datoteko enkrat testno prevedete z opcijo **draft**, kar vam pokaže predolge vrstice),

- če ste vključili pravilne verzije slik, predvsem pri grafih in diagramih vektorske (.pdf ali .eps) in ne rasterskih (npr. .jpg ali .png),
- če ste naslovili vse napake in opozorila, ki se lahko pojavijo pri tvorjenju dokumenta (bodite pozorni predvsem na manjkajoče citate, ki jih lahko poiščete tudi z iskanjem vprašajev v končnem dokumentu).

Literatura

Ostala literatura

- [1] V: *Encyclopedic Dictionary of Archaeology*. Springer International Publishing, 2021, str. 1150–1150. ISBN: 9783030582920. DOI: 10.1007/978-3-030-58292-0_180189. URL: http://dx.doi.org/10.1007/978-3-030-58292-0_180189.
- [2] Nicola Dragoni in sod. „Microservices: Yesterday, Today, and Tomorrow“. V: *Present and Ulterior Software Engineering*. Springer International Publishing, 2017, str. 195–216. ISBN: 9783319674254. DOI: 10.1007/978-3-319-67425-4_12.
- [3] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2015. ISBN: 9781491950357.
- [4] The PostgreSQL Global Development Group. *About PostgreSQL*. Dostopano: 21. 6. 2026. 2025. URL: <https://www.postgresql.org/about/>.
- [5] The PostgreSQL Global Development Group. *PostgreSQL Documentation: What Is PostgreSQL?* Dostopano: 21. 6. 2026. 2025. URL: <https://www.postgresql.org/docs/current/intro-what-is.html>.